SLOT 10

RESPONSIVE UI DESIGN TRONG FLUTTER

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- LayoutBuilder & MediaQuery
- Responsive UI Design
- Responsive Refactor

1 LayoutBuilder & MediaQuery

(Hiểu màn hình – hiểu không gian trước khi vẽ UI)

Khái niệm cốt lõi

`MediaQuery` giúp **lấy thông tin toàn bộ màn hình thiết bị**
(chiều rộng, chiều cao, orientation, textScale, padding…)

`LayoutBuilder` giúp **biết được không gian thực tế widget đang được phép sử dụng**

Responsive **không phải** là "lấy width màn hình rồi chia số bừa"

Sinh viên bắt buộc phải hiểu đúng

❌ *Sai lầm phổ biến*: luôn dùng `MediaQuery.of(context).size.width` cho mọi widget

✅ *Đúng bản chất*:

MediaQuery → nhìn từ *thiết bị*

LayoutBuilder → nhìn từ *cha cấp phát bao nhiêu chỗ cho con*

Mục tiêu học tập cụ thể

Biết khi nào **nên dùng MediaQuery**, khi nào **bắt buộc phải dùng LayoutBuilder**

Tạo được UI **không vỡ** khi:

mobile nhỏ
mobile lớn
tablet dọc / ngang

Ý nghĩa sư phạm (1–2 dòng)

Đây là bước đầu tiên giúp sinh viên **thoát khỏi tư duy "code theo máy của mình"**

Là nền tảng để hiểu mọi kỹ thuật responsive phía sau

2 Responsive UI Design

(Thiết kế UI thay đổi theo kích thước màn hình)

Khái niệm cốt lõi

Responsive UI là **thay đổi bố cục**, không chỉ thay đổi kích thước

UI phải **thích nghi**, không phải "co kéo cho vừa"

Sinh viên bắt buộc phải hiểu đúng

❌ Responsive ≠ dùng `SizedBox(width: screenWidth * 0.3)`

❌ Responsive ≠ fix cứng height / width

✅ Responsive =

thay đổi **layout (Row ↔ Column)**
thay đổi **số cột / cách sắp xếp**
ưu tiên **Flexible, Expanded, Wrap, Grid**

Mục tiêu học tập cụ thể

Thiết kế được UI:

cùng 1 màn hình nhưng layout khác nhau trên:

mobile nhỏ
mobile lớn
tablet

Biết chia breakpoint đơn giản:

< 600px

PhucPT10@fpt.edu.vn

600–1024px
1024px

Ý nghĩa sư phạm (1–2 dòng)

Giúp sinh viên hiểu rằng **UI là tư duy thiết kế**, không chỉ là code widget
Chuẩn bị tư duy làm app **dùng được lâu dài và chuyên nghiệp**

3 Responsive Refactor (Tái cấu trúc UI để không "vá lỗi")

Khái niệm cốt lõi

Responsive Refactor = **viết lại UI có chiến lược**, không sửa chắp vá
Tách UI thành:
- widget nhỏ
- layout theo kích thước màn hình

Sinh viên bắt buộc phải hiểu đúng

❌ Không nên nhét hết responsive logic vào 1 widget khổng lồ
❌ Không if/else layout một cách hỗn loạn
✅ Nên:
- tách widget theo vai trò
- mỗi widget chịu trách nhiệm responsive ở mức hợp lý

Mục tiêu học tập cụ thể

Refactor được:
- một màn hình đang vỡ layout
- thành UI rõ ràng, dễ mở rộng

Biết cách:
- đọc code cũ
- xác định điểm "không responsive"
- sửa đúng tư duy, không phá cấu trúc

Ý nghĩa sư phạm (1–2 dòng)

Giúp sinh viên vượt qua giai đoạn:
"App chạy được nhưng càng sửa càng rối"
Đây là kỹ năng **bắt buộc khi làm dự án thật và đi làm**

🎯 KẾT QUẢ SAU SLOT 10

Sau khi học xong Slot 10, sinh viên **tối thiểu phải đạt được**:

Hiểu **vì sao UI bị vỡ**, không còn sửa theo cảm giác
Biết chọn **đúng công cụ responsive**
Thiết kế UI **chạy ổn trên nhiều kích thước màn hình**
Có nền tảng để làm:
- app production
- tablet
- app dùng lâu dài

📚 TÀI LIỆU THAM KHẢO UY TÍN (DỄ TIẾP CẬN)

Flutter Official – Adaptive & Responsive Design
👉 https://docs.flutter.dev/ui/adaptive-responsive

LayoutBuilder Documentation
👉 https://api.flutter.dev/flutter/widgets/LayoutBuilder-class.html

MediaQuery Explained (Flutter)
👉 https://docs.flutter.dev/cookbook/design/orientation

Flutter Responsive UI – Practical Examples (Medium)
👉 Tìm với từ khóa: *"Flutter responsive layout for beginners"*

PhucPT10@fpt.edu.vn

YouTube – Flutter Responsive UI Tutorial

👉 Kênh gợi ý: *Flutterly*, *Reso Coder*, *FilledStacks*

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 NHÓM TƯ DUY RESPONSIVE UI (BẮT BUỘC PHẢI HIỂU TRƯỚC KHI CODE)

◈ Khái niệm quan trọng

Responsive UI trong Flutter là:

UI **thay đổi cách sắp xếp** theo không gian hiển thị
không phải chỉ là "co giãn cho vừa màn hình"

Flutter **vẽ UI dựa trên constraint (ràng buộc)**:

cha nói: *"con được dùng tối đa bao nhiêu chỗ"*
con tự quyết định vẽ trong giới hạn đó

👉 Responsive = **hiểu constraint trước, vẽ UI sau**

◈ Kỹ năng / thao tác BẮT BUỘC PHẢI THÀNH THẠO

Nhận diện được:

UI nào đang **phụ thuộc vào kích thước cố định**
UI nào đang **phụ thuộc vào constraint**

Biết thiết kế UI theo tư duy:

bố cục thay đổi (Row ↔ Column)
số lượng phần tử trên 1 hàng thay đổi

Không code UI theo "một màn hình duy nhất"

◈ Lỗi thường gặp của sinh viên mới

Nghĩ rằng:

Responsive = dùng `Expanded` là xong
Responsive = lấy `width * 0.3`

Code UI:

nhìn đẹp trên máy mình
vỡ hoàn toàn trên máy khác

👉 Nguyên nhân: **không hiểu Flutter layout hoạt động thế nào**

◈ Best Practices (Thực hành chuẩn)

Luôn tự hỏi trước khi code:

"Widget này được phép dùng bao nhiêu không gian?"

Ưu tiên:

Flexible / Expanded
Wrap / Grid

Không hard-code kích thước trừ khi **thật sự cần**

🎓 Ý nghĩa sư phạm

Đây là **bước sửa tư duy gốc**
Nếu không hiểu phần này, mọi kỹ thuật responsive phía sau đều chỉ là vá lỗi

2 NHÓM MEDIAQUERY (HIỂU MÀN HÌNH THIẾT BỊ)

◈ Khái niệm quan trọng

`MediaQuery` dùng để **lấy thông tin toàn bộ màn hình thiết bị**

width, height
orientation (dọc / ngang)
padding, safe area

MediaQuery **không biết widget đang nằm ở đâu**

👉 MediaQuery = **nhìn từ góc độ thiết bị**, không phải từ layout cha

PhucPT10@fpt.edu.vn

◈ Kỹ năng / thao tác BẮT BUỘC PHẢI THÀNH THẠO

Lấy thông tin màn hình:

- `MediaQuery.of(context).size`
- `orientation`

Sử dụng MediaQuery để:

- chia breakpoint đơn giản (mobile / tablet)
- thay đổi layout **cấp màn hình**

Kết hợp MediaQuery với:

- Scaffold
- App layout tổng

◈ Lỗi thường gặp của sinh viên mới

Dùng MediaQuery:

- trong widget con nhỏ
- để quyết định layout chi tiết

Dùng MediaQuery ở sai `context`

Lạm dụng:

- `width * 0.x` → dễ vỡ layout

👉 Nguyên nhân: **nhầm MediaQuery với LayoutBuilder**

◈ Best Practices (Thực hành chuẩn)

Dùng MediaQuery cho:

- layout **toàn màn hình**
- quyết định **dạng layout lớn**

Không dùng MediaQuery để:

- đo kích thước widget con

🎓 Ý nghĩa sư phạm

Giúp sinh viên hiểu:

"Thiết bị là thiết bị – layout là layout"

Tránh tư duy sai phổ biến: *"cứ lấy width màn hình là đúng"*

3 NHÓM LAYOUTBUILDER (HIỂU KHÔNG GIAN THỰC TẾ WIDGET)

◈ Khái niệm quan trọng

`LayoutBuilder` cho biết:

- widget **đang được phép dùng bao nhiêu không gian**

Dựa vào `BoxConstraints`:

- maxWidth
- maxHeight

👉 LayoutBuilder = **đọc constraint của widget cha**

◈ Kỹ năng / thao tác BẮT BUỘC PHẢI THÀNH THẠO

Sử dụng LayoutBuilder để:

- rẽ nhánh layout theo `maxWidth`
- thay đổi:
  - Row ↔ Column
  - số cột Grid

Kết hợp LayoutBuilder với:

- Card
- Item list
- Component UI tái sử dụng

PhucPT10@fpt.edu.vn

◈ Lỗi thường gặp của sinh viên mới

Không dùng LayoutBuilder, chỉ dùng MediaQuery
Nghĩ rằng:
    LayoutBuilder chỉ dùng cho màn hình lớn
Viết logic responsive:
    rối
    khó đọc
    nhét hết vào 1 widget

◈ Best Practices (Thực hành chuẩn)

Dùng LayoutBuilder cho:
    widget con
    component reusable
Rẽ nhánh layout **dựa trên constraint**, không dựa vào cảm giác

Ý nghĩa sư phạm

Đây là kỹ thuật giúp UI:
    ổn định
    tái sử dụng
    không phụ thuộc thiết bị
Là bước chuyển từ "code demo" sang "code app thật"

4 NHÓM RESPONSIVE REFACTOR (TỪ UI CỐ ĐỊNH → RESPONSIVE)

◈ Khái niệm quan trọng

Responsive Refactor là:
    sửa cấu trúc UI
    không phải thêm vài dòng if/else
Mục tiêu:
    UI rõ ràng
    dễ mở rộng
    không phá code cũ

◈ Kỹ năng / thao tác BẮT BUỘC PHẢI THÀNH THẠO

Đọc UI cũ và xác định:
    chỗ hard-code
    chỗ phụ thuộc kích thước
Tách UI thành:
    widget nhỏ
    layout theo vai trò
Refactor từng bước:
    không làm một lần quá lớn

◈ Lỗi thường gặp của sinh viên mới

Vá lỗi responsive:
    chỗ nào vỡ thì sửa chỗ đó
Nhét toàn bộ responsive logic:
    vào 1 widget lớn
Sau refactor:
    code khó đọc hơn ban đầu

◈ Best Practices (Thực hành chuẩn)

Refactor **càng sớm càng tốt**

PhucPT10@fpt.edu.vn

Tách:

layout

logic

Mỗi widget chịu trách nhiệm responsive ở **mức hợp lý**

Ý nghĩa sư phạm

Giúp sinh viên:

"Không sợ code cũ – biết cách sửa đúng"

Là kỹ năng **bắt buộc khi làm dự án lớn, làm việc nhóm**

TỔNG KẾT TRỌNG TÂM BẮT BUỘC

KIẾN THỨC CỐT LÕI (PHẢI NẮM)

Tư duy responsive UI

MediaQuery vs LayoutBuilder

Constraint-based layout

Tránh hard-code kích thước

KIẾN THỨC NÂNG CAO (CHỈ ĐỊNH HƯỚNG)

Breakpoint phức tạp

Adaptive layout chuyên sâu

Responsive cho desktop / web

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

1 RESPONSIVE UI ĐƯỢC ÁP DỤNG NHƯ THẾ NÀO TRONG DỰ ÁN FLUTTER THỰC TẾ?

Responsive UI trong app thật không phải để "đẹp hơn"

Mà để giải quyết **3 vấn đề sống còn**:

UI **không vỡ** trên nhiều thiết bị
Người dùng **dễ thao tác** trên màn hình khác nhau
Dev **không phải refactor lại toàn bộ UI** khi app mở rộng

Vai trò thật sự của từng kỹ thuật trong dự án

MediaQuery – Nhìn app từ góc độ thiết bị

Dùng để:

phân biệt **mobile / tablet**
xử lý **orientation (dọc – ngang)**
quyết định layout **cấp màn hình**

Không dùng để xử lý chi tiết từng widget

👉 MediaQuery giúp **xác định bối cảnh sử dụng app**

LayoutBuilder – Hiểu widget đang được cấp bao nhiêu không gian

Dùng để:

xử lý layout **bên trong màn hình**
viết widget **tái sử dụng**

Là công cụ chính để:

tránh vỡ layout
giữ UI ổn định khi đặt widget ở nhiều nơi khác nhau

👉 LayoutBuilder giúp UI **độc lập với thiết bị**

Responsive Refactor – Giảm đau khi app phát triển

Trong app thật:

UI **luôn thay đổi**
thêm tính năng → layout dễ vỡ

Refactor responsive sớm giúp:

giữ code sạch
mở rộng nhanh
tránh sửa hàng loạt về sau

👉 Responsive không làm chậm, **nó làm app sống lâu**

⚠ Nhấn mạnh

❌ Responsive UI **không phải** tính pixel thủ công
❌ Không phải `width * 0.3`
✅ Là:

tư duy constraint-based
thiết kế layout thích ứng

2 CÁC TÌNH HUỐNG THỰC TẾ (ỨNG DỤNG SLOT 10)

TÌNH HUỐNG 1: APP SINH VIÊN – MỖI MÁY MỖI KIỂU

Bối cảnh thực tế

App danh sách sinh viên
Sinh viên dùng:

máy nhỏ
máy lớn

PhucPT10@fpt.edu.vn

tablet giá rẻ

Vấn đề gặp phải

List item:

bị tràn text

nút quá nhỏ

Grid hiển thị:

quá to trên máy lớn

quá chật trên máy nhỏ

Cách áp dụng kỹ thuật Slot 10

MediaQuery:

xác định:

mobile hay tablet

LayoutBuilder:

quyết định:

layout item dạng Row hay Column

GridView:

thay đổi `crossAxisCount` theo `maxWidth`

Mục tiêu đạt được

UI:

không vỡ

dễ đọc

App dùng ổn trên nhiều thiết bị

Sinh viên hiểu:

"Không phải máy lỗi – là layout lỗi"

TÌNH HUỐNG 2: APP NỘI BỘ – DÙNG CHO ĐIỆN THOẠI & TABLET

Bối cảnh thực tế

App quản lý nội bộ

Nhân viên dùng:

điện thoại khi di chuyển

tablet khi làm việc tại bàn

Vấn đề gặp phải

Layout mobile:

lên tablet nhìn quá trống

Các nút:

xa nhau

thao tác chậm

Cách áp dụng kỹ thuật Slot 10

MediaQuery:

phát hiện tablet

LayoutBuilder:

chia layout:

1 cột (mobile)

2 cột (tablet)

Row / Column + Expanded:

tận dụng không gian ngang

PhucPT10@fpt.edu.vn

Mục tiêu đạt được

UI:

- tận dụng tốt màn hình lớn
- thao tác nhanh hơn

App:

- không cần viết 2 phiên bản

TÌNH HUỐNG 3: MVP DEMO – TRÌNH BÀY TRÊN MÀN HÌNH LỚN

Bối cảnh thực tế

Demo MVP cho khách hàng

Trình chiếu:

- màn hình lớn
- tablet ngang

Vấn đề gặp phải

UI mobile:

- bị kéo giãn
- nhìn thiếu chuyên nghiệp

Text:

- quá nhỏ
- bố cục lỏng lẻo

Cách áp dụng kỹ thuật Slot 10

MediaQuery:

- xác định orientation ngang

LayoutBuilder:

- chuyển layout:
  - stack dọc → layout ngang

GridView:

- tăng số cột

Tách widget:

- header
- content
- action

Mục tiêu đạt được

MVP:

- nhìn "như app thật"
- tạo ấn tượng tốt

Không phải sửa gấp trước buổi demo

3 CASE STUDY – RESPONSIVE REFACTOR THÀNH CÔNG

Bối cảnh dự án

App sinh viên

UI ban đầu:

- code cứng
- chỉ test trên 1 máy

Vấn đề ban đầu

Text tràn

Grid vỡ

PhucPT10@fpt.edu.vn

Code UI:
dài
khó sửa

🛠 Cách refactor

Bước 1: Dùng MediaQuery
phân loại mobile / tablet
Bước 2: Đưa LayoutBuilder vào widget con
xử lý layout item
Bước 3: Refactor:
Row cố định → layout linh hoạt
Grid cố định → Grid responsive

✅ Kết quả đạt được

UI:
ổn định trên nhiều thiết bị
Code:
dễ đọc
dễ mở rộng
Thêm tính năng mới:
không phá layout cũ

4 BÀI HỌC RÚT RA – LESSONS LEARNED

🔑 Điều quan trọng nhất khi làm Responsive UI

Hiểu không gian hiển thị trước khi vẽ UI

❌ Sai lầm phổ biến sinh viên hay mắc

Lạm dụng MediaQuery
Chia kích thước theo %
Nghĩ Responsive = Expanded

✅ Nguyên tắc cần duy trì lâu dài

Luôn tư duy **constraint-based**
Không hard-code layout
Refactor sớm
Layout rõ ràng hơn là "mẹo vặt"

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

PHẦN A. NỀN TẢNG TƯ DUY RESPONSIVE UI TRONG FLUTTER

I. VẤN ĐỀ UI ĐA THIẾT BỊ – WHY RESPONSIVE?

◈ UI cố định là gì?

UI cố định là UI:

- được thiết kế **cho 1 kích thước màn hình cụ thể**
- thường hard-code:
  - width
  - height
  - số cột

Ví dụ phổ biến:

- "Máy em chạy đẹp rồi"
- "Em test trên emulator Pixel 4"

👉 UI cố định **không sai về mặt cú pháp**, nhưng **sai về tư duy sản phẩm**

◈ Vì sao UI chạy đúng trên máy này nhưng vỡ trên máy khác?

Các nguyên nhân cốt lõi:

Kích thước màn hình khác nhau
- mobile nhỏ / mobile lớn / tablet

Tỉ lệ màn hình khác nhau
- 16:9, 18:9, 20:9…

Orientation
- dọc → ngang

Mật độ hiển thị (pixel density)
- cùng kích thước nhưng hiển thị khác

👉 Flutter **không đảm bảo** UI bạn viết sẽ giống nhau trên mọi thiết bị

◈ Sinh viên thường hiểu sai ở đâu?

Nghĩ rằng:

- "Cùng là 360px thì giống nhau"
- "Lấy width màn hình chia % là đủ"

Tư duy:

- layout = pixel
- không gian hiển thị là cố định

👉 Đây là **tư duy web / design tĩnh**, không phù hợp Flutter

◈ So sánh với HTML/CSS (để sinh viên dễ liên hệ)

HTML/CSS:

- responsive dựa vào:
  - viewport
  - media query

Flutter:

- responsive dựa vào:
  - **constraint**
  - widget tree

👉 Flutter **không layout theo viewport**, mà theo **cây widget**

🎓 Ý nghĩa sư phạm

Sinh viên cần hiểu:

> "UI vỡ không phải do Flutter khó, mà do mình thiết kế sai tư duy"

Đây là lý do **Responsive UI là bắt buộc**, không phải kỹ thuật nâng cao

PhucPT10@fpt.edu.vn

II. TƯ DUY LAYOUT ENGINE CỦA FLUTTER

◈ Flutter layout hoạt động theo 3 bước

Flutter layout **luôn tuân theo quy trình này**:

Parent → Constraints

Cha nói: *"Con được dùng tối đa bao nhiêu không gian"*

Child → Size

Con chọn kích thước trong giới hạn đó

Parent → Position

Cha đặt con vào vị trí cụ thể

👉 Không có bước nào là "vẽ theo pixel tuyệt đối"

◈ Vì sao Flutter không layout theo pixel?

Flutter:

chạy trên **rất nhiều thiết bị**

kích thước & tỉ lệ khác nhau

Nếu layout theo pixel:

UI sẽ **vỡ hàng loạt**

👉 Constraint-based layout giúp Flutter:

linh hoạt

ổn định

mở rộng lâu dài

◈ Vì sao Expanded / Flexible tồn tại?

`Expanded` / `Flexible`:

cho phép widget **chiếm không gian còn lại**

thay vì ép kích thước cứng

Đây là cách Flutter:

giải quyết layout động

tránh hard-code

◈ Vì sao hard-code width / height là sai tư duy?

Hard-code:

phá vỡ constraint

phụ thuộc thiết bị

Khi UI phình to:

code khó sửa

layout khó kiểm soát

👉 Hard-code = **đánh đổi tương lai của app**

🎓 Ý nghĩa sư phạm

Nếu sinh viên **hiểu được 3 bước layout**, mọi vấn đề responsive sau này đều có lời giải

Đây là nền tảng cho:

LayoutBuilder

Custom layout

Adaptive UI

III. MEDIAQUERY – TIẾP CẬN THEO KÍCH THƯỚC MÀN HÌNH

◈ MediaQuery là gì?

`MediaQuery` cung cấp **thông tin về thiết bị đang chạy app**

Bao gồm:

size (width / height)

PhucPT10@fpt.edu.vn

orientation
padding (safe area)
viewInsets (bàn phím)

◈ Khi nào nên dùng MediaQuery?

Khi cần:

phân biệt **mobile / tablet**
xử lý **layout cấp màn hình**
thay đổi UI theo orientation

👉 MediaQuery dùng cho **bối cảnh sử dụng app**

◈ Khi nào KHÔNG nên dùng MediaQuery?

Không nên dùng khi:

xử lý layout widget con
viết component tái sử dụng

Vì MediaQuery:

không biết widget đang nằm ở đâu
chỉ biết toàn màn hình

◈ Lỗi phổ biến sinh viên hay mắc

Lạm dụng MediaQuery:

dùng ở mọi widget

Chia kích thước:

`width * 0.x`

Phụ thuộc:

screen size toàn cục

👉 Kết quả:

UI khó kiểm soát
refactor rất đau

🎓 Ý nghĩa sư phạm

MediaQuery **không sai**
Sai là **dùng sai ngữ cảnh**
Sinh viên cần hiểu:
"Thiết bị ≠ layout của widget"

IV. LAYOUTBUILDER – TƯ DUY DỰA TRÊN CONSTRAINT

◈ LayoutBuilder là gì?

`LayoutBuilder` cung cấp:

`BoxConstraints`
thông tin không gian **widget được phép sử dụng**

Bao gồm:

maxWidth
maxHeight

◈ Khác gì MediaQuery?

MediaQuery:

nhìn từ **thiết bị**

LayoutBuilder:

nhìn từ **widget cha**

👉 Đây là khác biệt **cốt lõi nhất**

PhucPT10@fpt.edu.vn

◈ Vì sao LayoutBuilder phù hợp widget con?

Widget con:
- không biết mình nằm trong layout nào

LayoutBuilder:
- cho biết constraint thực tế
- giúp widget:
  - linh hoạt
  - tái sử dụng

◈ Vì sao LayoutBuilder giúp code tốt hơn?

Ít phụ thuộc thiết bị
Dễ refactor
Ít bug layout khi tái sử dụng

🎓 Ý nghĩa sư phạm

LayoutBuilder là bước chuyển:
từ "code cho màn hình" → "code cho không gian"
Đây là tư duy **developer chuyên nghiệp**

V. SO SÁNH MEDIAQUERY vs LAYOUTBUILDER (TRỌNG TÂM)

◈ So sánh theo tư duy

MediaQuery:
- thiết bị-centric

LayoutBuilder:
- layout-centric

◈ Dùng sai → hậu quả

MediaQuery cho widget con:
- layout khó đoán

LayoutBuilder cho toàn màn hình:
- code phức tạp không cần thiết

◈ Kết luận

❌ Không có công cụ tốt nhất
✅ Chỉ có công cụ **đúng ngữ cảnh**

PHẦN B. RESPONSIVE UI DESIGN & RESPONSIVE REFACTOR

VI. RESPONSIVE UI DESIGN TRONG FLUTTER

◈ Responsive UI là gì (đúng nghĩa)?

Responsive UI:
- **thay đổi bố cục**
- không chỉ thay đổi kích thước

◈ Responsive ≠ Expanded

Expanded chỉ là:
- công cụ chia không gian

Không quyết định:
- layout tổng thể

PhucPT10@fpt.edu.vn

◈ Nguyên tắc responsive cơ bản

Ưu tiên:

- Column → mobile
- Row / Grid → màn hình lớn

Kết hợp:

- Flexible
- Wrap
- GridView

🎓 Ý nghĩa sư phạm

Giúp sinh viên:

- thiết kế UI có tư duy
- không phụ thuộc demo

VII. RESPONSIVE REFACTOR – TỪ UI CỐ ĐỊNH → UI LINH HOẠT

◈ Khi nào cần refactor?

UI vỡ
Code khó mở rộng
Thêm tính năng là lỗi layout

◈ Quy trình refactor

Nhận diện hard-code
Phân tích constraint
Chọn MediaQuery / LayoutBuilder
Test nhiều kích thước

🎓 Ý nghĩa sư phạm

Refactor là kỹ năng **bắt buộc**
Không có app thật nào "đúng ngay từ đầu"

VIII. LỖI PHỔ BIẾN KHI LÀM RESPONSIVE UI

UI vẫn vỡ → tư duy sai constraint
LayoutBuilder sai chỗ → dùng sai ngữ cảnh
Grid sai → số cột cố định
Text overflow → không xét không gian
Nested layout → không kiểm soát constraint

IX. LIÊN HỆ DỰ ÁN THẬT & MODULE TIẾP THEO

Responsive UI ảnh hưởng trực tiếp đến:

- UX
- maintainability
- scalability

Slot 10 là nền tảng cho:

- adaptive UI
- app đa nền tảng

X. TỔNG KẾT TƯ DUY RESPONSIVE UI

🔑 Điều quan trọng nhất

Hiểu không gian trước khi vẽ UI

✕ 3 sai lầm cần tránh

Pixel cứng
MediaQuery everywhere

PhucPT10@fpt.edu.vn

Responsive = Expanded

🚀 Chuẩn bị cho bước tiếp theo

Widget nâng cao

Layout phức tạp

App Flutter quy mô lớn

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. KỸ THUẬT: MEDIAQUERY

Kinh nghiệm thực tế khi dùng MediaQuery trong dự án Flutter

MediaQuery xuất hiện ở đâu trong app thật?

Màn hình chính (Home / Dashboard)

Quyết định:

- mobile hay tablet
- layout dọc hay ngang

Xử lý:

- SafeArea
- bàn phím che UI
- orientation change

Điểm KHÔNG giống tutorial

Tutorial thường:

- lấy `MediaQuery.of(context).size.width`
- chia % cho mọi widget

App thật:

- UI phức tạp hơn
- widget lồng nhiều cấp
- MediaQuery dùng sai chỗ → layout khó đoán

Tình huống thực tế

Bối cảnh: App nội bộ, có form nhập liệu

Vấn đề:

- Bàn phím bật lên → UI bị đẩy vỡ

Vì sao chọn MediaQuery:

- Cần biết `viewInsets.bottom`
- Layout cấp màn hình

Lỗi thường gặp khi dùng MediaQuery

Lỗi 1: Lạm dụng MediaQuery cho widget con

Mô tả:

- UI con tự lấy width màn hình
- Đặt ở đâu cũng "to bất thường"

Nguyên nhân gốc:

- Tư duy: *widget biết màn hình*
- Trong khi widget **chỉ biết cha nó**

Khắc phục:

- Chuyển sang LayoutBuilder

Phòng tránh:

- MediaQuery chỉ dùng ở **cấp màn hình**

Lỗi 2: Chia kích thước theo %

Mô tả:

```
width * 0.3, height * 0.2
```

Hậu quả:

- vỡ layout
- khó maintain

Nguyên nhân:

- Tư duy pixel

Khắc phục:

PhucPT10@fpt.edu.vn

Dùng Expanded / Flexible

Phòng tránh:

Không tính toán kích thước thủ công

Mẹo lập trình Responsive với MediaQuery

Chỉ dùng MediaQuery cho:

- breakpoint lớn
- orientation

Không dùng cho:

- layout item
- widget reusable

Luôn kiểm tra:

- mobile nhỏ
- mobile lớn
- tablet (cơ bản)

Best Practices – MediaQuery

✅ Dùng đúng ngữ cảnh

❌ Không dùng như "thước đo vạn năng"

⚠️ MediaQuery **không thay thế LayoutBuilder**

Bài học rút ra

MediaQuery trả lời câu hỏi: "Thiết bị là gì?" – không phải "widget rộng bao nhiêu?"

2. KỸ THUẬT: LAYOUTBUILDER

Kinh nghiệm thực tế khi dùng LayoutBuilder

LayoutBuilder xuất hiện ở đâu trong app thật?

- Card
- Item list
- Component tái sử dụng
- Widget được nhúng nhiều nơi

Điểm KHÔNG giống tutorial

Tutorial:

- demo 1 widget đơn giản

App thật:

- LayoutBuilder lồng trong:
  - ListView
  - GridView
  - Column phức tạp

Tình huống thực tế

Bối cảnh: Card sản phẩm

Vấn đề:

- Mobile: cần layout dọc
- Tablet: cần layout ngang

Vì sao chọn LayoutBuilder:

- Card không biết mình nằm ở đâu
- Chỉ constraint mới đúng

⚠ Lỗi thường gặp khi dùng LayoutBuilder

✕ *Lỗi 1: Không dùng LayoutBuilder cho widget con*

Mô tả:
Widget con phụ thuộc MediaQuery
Nguyên nhân:
Không hiểu constraint
Khắc phục:
Bọc widget con bằng LayoutBuilder
Phòng tránh:
Widget reusable → LayoutBuilder

✕ *Lỗi 2: Logic responsive quá phức tạp*

Mô tả:
nhiều if/else
Nguyên nhân:
Không tách widget
Khắc phục:
Tách layout theo vai trò
Phòng tránh:
Widget nhỏ – trách nhiệm rõ

💡 Mẹo dùng LayoutBuilder hiệu quả

Dùng `constraints.maxWidth` làm breakpoint
Breakpoint đơn giản:
< 600
≥ 600
Không dùng magic number rải rác

🛡 Best Practices – LayoutBuilder

Widget càng tái sử dụng → càng nên dùng LayoutBuilder
Ít phụ thuộc thiết bị → code sống lâu hơn

🎓 Bài học rút ra

LayoutBuilder giúp widget "sống đúng chỗ", không phụ thuộc màn hình

◈ 3. RESPONSIVE VỚI ROW / COLUMN / EXPANDED / FLEXIBLE

☑ Kinh nghiệm thực tế

App thật:
layout thay đổi theo chiều ngang
Row / Column:
quyết định UX trực tiếp

⚠ Lỗi thường gặp

✕ *Nhầm Responsive = Expanded*

Mô tả:
Dùng Expanded khắp nơi
Nguyên nhân:
Hiểu sai bản chất
Khắc phục:
Expanded chỉ chia không gian
Phòng tránh:

PhucPT10@fpt.edu.vn

Responsive là **thay đổi layout**

Tips

Mobile:

Column

Tablet:

Row + Expanded

Luôn test text dài

Bài học rút ra

Expanded giúp chia chỗ – không giúp quyết định bố cục

4. GRIDVIEW RESPONSIVE

Lỗi phổ biến

`crossAxisCount` cố định

Grid quá to / quá nhỏ

Khắc phục

Dùng:

MediaQuery cho toàn màn hình

LayoutBuilder cho grid con

Điều chỉnh số cột theo width

Bài học rút ra

GridView không responsive nếu số cột cố định

5. RESPONSIVE REFACTOR – TỪ UI CỐ ĐỊNH → LINH HOẠT

Sai lầm phổ biến

Vá từng lỗi UI

Không sửa tư duy layout

Quy trình chuẩn

Xác định hard-code

Phân tích constraint

Chọn đúng công cụ

Test đa thiết bị

Bài học rút ra

Refactor sớm rẻ hơn sửa cháy

TỔNG KẾT

Responsive **không phải mẹo**

Là:

tư duy layout

kỹ năng làm app thật

Nếu nhớ **3 điều**:

Hiểu constraint

MediaQuery ≠ LayoutBuilder

Không hard-code UI

PhucPT10@fpt.edu.vn

EXERCISE

BÀI TẬP TỔNG HỢP

RESPONSIVE DASHBOARD APP – FLUTTER

1 TỔNG QUAN DỰ ÁN

Tên dự án

Responsive Dashboard App

Mô tả dự án

Đây là một ứng dụng Flutter hiển thị **Dashboard thông tin**, bao gồm:

Header (tiêu đề + icon)
Các thẻ thống kê (Summary Cards)
Danh sách / Grid hiển thị item

Toàn bộ UI **tự thích ứng** trên:

📱 Điện thoại nhỏ
📱 Điện thoại lớn
📲 Tablet cơ bản

👉 **Yêu cầu quan trọng**:

UI **không vỡ layout**, **không tràn**, **không phụ thuộc kích thước cố định**.

Mục tiêu học tập (rất quan trọng)

Sau khi hoàn thành dự án này, sinh viên **bắt buộc đạt được**:

Hiểu **vì sao UI vỡ khi đổi thiết bị**
Biết **khi nào dùng MediaQuery, khi nào dùng LayoutBuilder**
Thực hành **Responsive Refactor đúng tư duy**
Thoát khỏi thói quen nguy hiểm:

```
width = 300
height = 120
```

👉 Đây là bước chuyển từ **code bài tập → code app thật**

Đối tượng phù hợp

Sinh viên đã học xong **Flutter Fundamentals (Module 2)**
Trình độ **mới học**
Đã biết:

- Row / Column
- Expanded
- Scaffold
- StatelessWidget

2 CHỨC NĂNG CHÍNH (GẮN TRỰC TIẾP VỚI RESPONSIVE)

Màn hình Dashboard gồm

Header

- Tiêu đề
- Icon

Summary Cards

- Hiển thị thông tin thống kê

Grid danh sách item

Liên hệ kỹ thuật Responsive

Trong dự án này, sinh viên sẽ **trực tiếp xử lý**:

PhucPT10@fpt.edu.vn

Layout thay đổi theo **chiều rộng màn hình**
GridView **tự đổi số cột**
Text **không bị overflow**
Spacing **linh hoạt theo kích thước**

⚠️ **Không tập trung business logic**

👉 Chỉ tập trung **layout & responsive tư duy**

3 CÔNG NGHỆ & CÔNG CỤ

Flutter SDK
Dart
Android Studio / VS Code
Android Emulator (nhiều kích thước)
Material Widgets

4 CẤU TRÚC THƯ MỤC DỰ ÁN

```
lib/
 ├── main.dart
 ├── screens/
 │    └── dashboard_screen.dart
 ├── widgets/
 │    ├── summary_card.dart
 │    └── responsive_grid.dart
```

📌 Ý nghĩa sư phạm

Tách widget nhỏ → **dễ responsive**
Tránh viết **500 dòng UI trong 1 file**
Refactor layout **không phá toàn màn hình**

5 LỘ TRÌNH THỰC HIỆN (TỔNG QUAN)

1.Tạo project Flutter
2. Dựng **UI cố định (cố tình sai)**
3. Chạy trên nhiều emulator → **thấy UI vỡ**
4. Phân tích nguyên nhân
5. Áp dụng MediaQuery (spacing, text)
6. Áp dụng LayoutBuilder (layout chính)
7. Responsive Refactor Summary Cards
8. Responsive GridView
9. Kiểm tra lại trên nhiều thiết bị
10.Tổng kết & rút kinh nghiệm

👉 **Bước 7–8 là bắt buộc** (Responsive Refactor)

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

0. Chuẩn bị ban đầu & cấu trúc thư mục

0.1. Tạo project

Mở terminal (hoặc Terminal trong VS Code / Android Studio):

```bash
flutter create responsive_dashboard
cd responsive_dashboard
```


PhucPT10@fpt.edu.vn

Chạy thử để chắc app chạy OK:

```bash
flutter run
```

Lúc này app chỉ là Flutter counter mặc định.

0.2. Tạo cấu trúc thư mục như yêu cầu

Trong thư mục `lib/`, bạn tạo thêm:

- thư mục `screens/`
- thư mục `widgets/`

Cấu trúc:

```
lib/
 ├── main.dart
 ├── screens/
 │    └── dashboard_screen.dart
 ├── widgets/
 │    ├── summary_card.dart
 │    └── responsive_grid.dart
```

Bạn có thể tạo trực tiếp trong VS Code:

- Click phải vào `lib` → **New Folder** → `screens`
- Click phải vào `lib` → **New Folder** → `widgets`
- Click phải vào `screens` → **New File** → `dashboard_screen.dart`
- Click phải vào `widgets` → **New File** → `summary_card.dart`
- Click phải vào `widgets` → **New File** → `responsive_grid.dart`

1. File `main.dart` – Kết nối tới DashboardScreen

1.1. Mục tiêu

- Thay `MyHomePage` mặc định bằng `DashboardScreen`.
- Đảm bảo project start đúng màn hình cần học responsive.

1.2. Code hoàn chỉnh cho `lib/main.dart`

lib/main.dartv1

```dart
import 'package:flutter/material.dart';
import 'screens/dashboard_screen.dart';

void main() {
  runApp(const ResponsiveDashboardApp());
}

class ResponsiveDashboardApp extends StatelessWidget {
  const ResponsiveDashboardApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Responsive Dashboard',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: false,
```


PhucPT10@fpt.edu.vn

```
      ),
      home: const DashboardScreen(),
    );
  }
}
```

1.3. Giải thích

`MaterialApp` là "khung" của app.

`home: const DashboardScreen()` nghĩa là khi chạy app, Flutter sẽ hiển thị widget `DashboardScreen`.

Mọi thứ chúng ta học về **responsive** sẽ nằm trong `DashboardScreen`.

2. BƯỚC 2 – TẠO UI CỐ ĐỊNH (CỐ TÌNH SAI)

2.1. Mục tiêu sư phạm

Kiểm chứng **thấy UI bị vỡ thật** → tạo động lực hiểu responsive.

Cố tình "hard-code" width/height.

2.2. Tạo `DashboardScreen` với layout sai

Trong `lib/screens/dashboard_screen.dart`:

lib/screens/dashboard_screen.dartv1

```
import 'package:flutter/material.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Responsive Dashboard (Version: Fixed UI -
Wrong)'),
      ),
      body: SingleChildScrollView(
        child: Column(
          children: [
            // Header / Summary cố tình sai (hard-code size)
            const SizedBox(height: 16),
            Container(
              width: 350,
              height: 120,
              color: Colors.blue,
              alignment: Alignment.center,
              child: const Text(
                'Summary (Fixed width 350, height 120)',
                style: TextStyle(color: Colors.white, fontSize: 20),
              ),
            ),
            const SizedBox(height: 16),

            // Row summary cards cố tình sai (hard-code size)
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
```


PhucPT10@fpt.edu.vn

```
                Container(
                  width: 160,
                  height: 120,
                  color: Colors.red,
                  alignment: Alignment.center,
                  child: const Text(
                    'Card 1\n160 x 120',
                    style: TextStyle(color: Colors.white),
                    textAlign: TextAlign.center,
                  ),
                ),
                const SizedBox(width: 16),
                Container(
                  width: 160,
                  height: 120,
                  color: Colors.green,
                  alignment: Alignment.center,
                  child: const Text(
                    'Card 2\n160 x 120',
                    style: TextStyle(color: Colors.white),
                    textAlign: TextAlign.center,
                  ),
                ),
              ],
            ),

            const SizedBox(height: 24),

            // Giả lập grid sản phẩm - cũng cố định
            SizedBox(
              height: 400,
              child: GridView.count(
                crossAxisCount: 3, // cố định, không responsive
                childAspectRatio: 1.2,
                children: List.generate(
                  8,
                  (index) => Container(
                    margin: const EdgeInsets.all(8),
                    color: Colors.orange,
                    alignment: Alignment.center,
                    child: Text(
                      'Item $index',
                      style: const TextStyle(color: Colors.white),
                    ),
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
```


PhucPT10@fpt.edu.vn

```
}
```

2.3. Thực hiện chạy & quan sát

Mở **emulator điện thoại nhỏ** (ví dụ: 360 x 640, Pixel 3a...)

Chạy:

```bash
flutter run
```

Quan sát:

3 cột `GridView` trên màn nhỏ → rất bé, khó đọc.

Row 2 card có thể bị tràn hoặc sát mép viền, thiếu khoảng cách.

Trên điện thoại nhỏ hơn nữa → có thể **tràn** (overflow).

Mở **tablet / device lớn** (hoặc chỉnh lại device width lớn trong emulator):

Card nhỏ xíu, nằm lệch, layout không "đẹp".

2.4. Tự đánh giá

"Tại sao layout này chỉ đẹp trên 1 kích thước?"

"Card có kích thước cố định 160 x 120 → khi màn nhỏ hơn thì sao?"

"Grid 3 cột cố định → nếu màn nhỏ xíu hoặc rất to thì sao?"

3. BƯỚC 3 – PHÂN TÍCH NGUYÊN NHÂN UI VỠ

3.1. Tự phát hiện vấn đề những dòng đang sai

Các đoạn gây vấn đề:

```dart
Container(
  width: 350,
  height: 120,
  ...
)

Container(
  width: 160,
  height: 120,
  ...
)

GridView.count(
  crossAxisCount: 3,
  ...
)
```

3.2. Giải thích tư duy constraint-based

Flutter **không layout theo pixel cố định** như Photoshop.

Mỗi widget nhận **constraints** từ cha:

`maxWidth`, `minWidth`, `maxHeight`, `minHeight`.

Khi bạn **ép** `width = 350` nhưng cái `Row` chứa nó chỉ được phép rộng 320 → **overflow**.

Câu hỏi checkpoint:

"Nếu đổi sang máy khác, kích thước màn hình khác, thì các con số 350, 160 này còn hợp lý không?"

4. BƯỚC 4 – DÙNG MEDIAQUERY CHO SPACING & TEXT

Ở bước này **chỉ dùng MediaQuery cho spacing & font size**, **không dùng cho mọi thứ**.

PhucPT10@fpt.edu.vn

4.1. Sửa DashboardScreen – thêm Header dùng MediaQuery

Ta bắt đầu refactor DashboardScreen sang **phiên bản có MediaQuery**, nhưng **chưa responsive layout**, chỉ responsive kích thước chữ + padding.

lib/screens/dashboard_screen.dartv2

```
import 'package:flutter/material.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // 1 Lấy kích thước màn hình
    final screenSize = MediaQuery.of(context).size;
    final screenWidth = screenSize.width;

    // 2 Quy ước vài tỷ lệ
    final horizontalPadding = screenWidth * 0.04; // 4% width
    final headerFontSize = screenWidth * 0.06;    // 6% width

    return Scaffold(
      appBar: AppBar(
        title: const Text('Responsive Dashboard (MediaQuery Step)'),
      ),
      body: SingleChildScrollView(
        child: Padding(
          padding: EdgeInsets.symmetric(horizontal: horizontalPadding),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const SizedBox(height: 16),

              // Header dùng MediaQuery cho fontSize
              Text(
                'Dashboard',
                style: TextStyle(
                  fontSize: headerFontSize.clamp(20, 32), // Giới hạn min/max
                  fontWeight: FontWeight.bold,
                ),
              ),

              const SizedBox(height: 16),

              // Vẫn để layout sai ở dưới (chưa refactor)
              Container(
                width: 350,
                height: 120,
                color: Colors.blue,
                alignment: Alignment.center,
                child: const Text(
                  'Summary (Still fixed size)',
                  style: TextStyle(color: Colors.white, fontSize: 20),
                ),
```


PhucPT10@fpt.edu.vn

```
            ),
            const SizedBox(height: 16),

            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Container(
                  width: 160,
                  height: 120,
                  color: Colors.red,
                  alignment: Alignment.center,
                  child: const Text(
                    'Card 1\n160 x 120',
                    style: TextStyle(color: Colors.white),
                    textAlign: TextAlign.center,
                  ),
                ),
                const SizedBox(width: 16),
                Container(
                  width: 160,
                  height: 120,
                  color: Colors.green,
                  alignment: Alignment.center,
                  child: const Text(
                    'Card 2\n160 x 120',
                    style: TextStyle(color: Colors.white),
                    textAlign: TextAlign.center,
                  ),
                ),
              ],
            ),

            const SizedBox(height: 24),

            SizedBox(
              height: 400,
              child: GridView.count(
                crossAxisCount: 3,
                childAspectRatio: 1.2,
                children: List.generate(
                  8,
                  (index) => Container(
                    margin: const EdgeInsets.all(8),
                    color: Colors.orange,
                    alignment: Alignment.center,
                    child: Text(
                      'Item $index',
                      style: const TextStyle(color: Colors.white),
                    ),
                  ),
                ),
              ),
            ),
```


PhucPT10@fpt.edu.vn

```
            ],
          ),
        ),
      ),
    );
  }
}
```

4.2. Giải thích

`MediaQuery.of(context).size.width` lấy **chiều rộng toàn màn hình** (không phải từng widget).

`horizontalPadding = screenWidth * 0.04` nghĩa là:

Màn nhỏ → padding nhỏ.

Màn to → padding lớn hơn, nhưng vẫn **tỉ lệ** với màn hình.

`.clamp(20, 32)` giúp font size **không quá nhỏ/quá lớn**:

Nếu tính ra < 20 → dùng 20.

Nếu > 32 → dùng 32.

Lưu ý sư phạm quan trọng:

Không dùng `MediaQuery` cho mọi thứ (width, height của card, v.v...)

Chỉ dùng để điều chỉnh "nhẹ" spacing, font size cho cảm giác dễ nhìn hơn.

5. BƯỚC 5 – DÙNG LAYOUTBUILDER CHO LAYOUT CHÍNH

Giờ ta bắt đầu chia layout theo **breakpoint**:

Màn nhỏ: summary cards xếp **dọc** (Column).

Màn lớn: summary cards xếp **ngang** (Row).

5.1. Ý tưởng

Bọc khu v►►c **summary cards** vào `LayoutBuilder`.

Dựa vào `constraints.maxWidth` để quyết định Column hay Row.

5.2. Tạo private widget `_SummarySection`

Để không làm `build` quá dài, ta tách phần summary thành widget riêng bên trong file `dashboard_screen.dart`.

lib/screens/dashboard_screen.dartv3

```
import 'package:flutter/material.dart';
import '../widgets/summary_card.dart';
import '../widgets/responsive_grid.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final screenSize = MediaQuery.of(context).size;
    final screenWidth = screenSize.width;

    final horizontalPadding = screenWidth * 0.04;
    final headerFontSize = screenWidth * 0.06;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Responsive Dashboard'),
      ),
```


PhucPT10@fpt.edu.vn

```
      body: SingleChildScrollView(
        child: Padding(
          padding: EdgeInsets.symmetric(horizontal: horizontalPadding),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const SizedBox(height: 16),

              Text(
                'Dashboard',
                style: TextStyle(
                  fontSize: headerFontSize.clamp(20, 32),
                  fontWeight: FontWeight.bold,
                ),
              ),

              const SizedBox(height: 16),

              // 1 Khu vực summary dùng LayoutBuilder
              const _SummarySection(),

              const SizedBox(height: 24),

              // 2 Khu vực grid sản phẩm tách ra widget riêng
              const ResponsiveProductGrid(),
            ],
          ),
        ),
      ),
    );
  }
}

class _SummarySection extends StatelessWidget {
  const _SummarySection();

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        // breakpoint đơn giản: < 600 là màn nhỏ
        final isNarrow = constraints.maxWidth < 600;

        // Tạo 3 summary cards làm ví dụ
        final cards = [
          const SummaryCard(
            title: 'Revenue',
            value: '\$12,300',
            color: Colors.blue,
          ),
          const SummaryCard(
            title: 'Orders',
            value: '320',
```


PhucPT10@fpt.edu.vn

```
            color: Colors.green,
          ),
          const SummaryCard(
            title: 'Customers',
            value: '89',
            color: Colors.purple,
          ),
        ];

        if (isNarrow) {
          // Màn nhỏ: xếp dọc
          return Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              ...cards.map((c) => Padding(
                    padding: const EdgeInsets.only(bottom: 12),
                    child: c,
                  )),
            ],
          );
        } else {
          // Màn rộng: xếp ngang, dùng Expanded (sẽ giải thích kỹ ở bước 6)
          return Row(
            children: [
              Expanded(child: cards[0]),
              const SizedBox(width: 12),
              Expanded(child: cards[1]),
              const SizedBox(width: 12),
              Expanded(child: cards[2]),
            ],
          );
        }
      },
    );
  }
}
```

Lúc này ta đang gọi `SummaryCard` & `ResponsiveProductGrid` – ở bước tiếp theo ta sẽ tạo 2 widget này.

6. BƯỚC 6 – RESPONSIVE REFACTOR SUMMARY CARD

6.1. Mục tiêu

Thay vì `Container(width: 160, height: 120)`, ta:

Cho `Row/Column` quyết định size bằng `Expanded`.

Bản thân `SummaryCard` **không hard-code chiều rộng**, chỉ set padding, border, v.v.

6.2. Tạo file `lib/widgets/summary_card.dart`

lib/widgets/summary_card.dartv1

```
import 'package:flutter/material.dart';

class SummaryCard extends StatelessWidget {
  final String title;
  final String value;
```


PhucPT10@fpt.edu.vn

```
final Color color;

const SummaryCard({
  super.key,
  required this.title,
  required this.value,
  required this.color,
});

@override
Widget build(BuildContext context) {
  // Dùng MediaQuery nhẹ nhàng cho fontSize bên trong card
  final screenWidth = MediaQuery.of(context).size.width;
  final titleFontSize = (screenWidth * 0.035).clamp(12, 18);
  final valueFontSize = (screenWidth * 0.05).clamp(18, 28);

  return Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: color.withOpacity(0.1),
      borderRadius: BorderRadius.circular(12),
      border: Border.all(color: color, width: 1),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // Tiêu đề
        Text(
          title,
          style: TextStyle(
            fontSize: titleFontSize,
            fontWeight: FontWeight.w500,
            color: color,
          ),
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
        ),

        const SizedBox(height: 8),

        // Giá trị chính
        Text(
          value,
          style: TextStyle(
            fontSize: valueFontSize,
            fontWeight: FontWeight.bold,
            color: Colors.black87,
          ),
        ),
      ],
    ),
  );
}
```


PhucPT10@fpt.edu.vn

```
}
```

6.3. Giải thích "Responsive Refactor cốt lõi"

- `SummaryCard` **không có** `width: ...`:
  - Nếu đặt trong `Expanded` của `Row` → nó tự chiếm phần **bằng nhau**.
  - Nếu đặt trong `Column` với `crossAxisAlignment.stretch` → nó full width theo Column.
- `Expanded` nằm ở cha (`_SummarySection`), không nằm trong `SummaryCard`.
- Đây chính là tư duy:

"Widget con không tự quyết định kích thước cố định.
Nó **tuân thủ constraints** do cha đưa."

Bạn có thể dừng lại để sinh viên tự so sánh:

- Card cũ: `Container(width: 160, height: 120, ...)`
- Card mới: `SummaryCard()` → không width/height.

7. BƯỚC 7 – GRIDVIEW RESPONSIVE THEO WIDTH

7.1. Mục tiêu

- `GridView` đổi số cột theo chiều rộng:
  - < 600: 2 cột
  - 600–900: 3 cột
  - 900: 4 cột
- Sử dụng `LayoutBuilder` để lấy `constraints.maxWidth` **của vùng grid**, không dùng MediaQuery toàn màn hình.

7.2. Tạo `lib/widgets/responsive_grid.dart`

lib/widgets/responsive_grid.dartv1

```
import 'package:flutter/material.dart';

class ResponsiveProductGrid extends StatelessWidget {
  const ResponsiveProductGrid({super.key});

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final double maxWidth = constraints.maxWidth;

        int crossAxisCount;
        if (maxWidth < 600) {
          crossAxisCount = 2;
        } else if (maxWidth < 900) {
          crossAxisCount = 3;
        } else {
          crossAxisCount = 4;
        }

        const double childAspectRatio = 1.1;

        return Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
```


PhucPT10@fpt.edu.vn

```
              'Products (width: ${maxWidth.toStringAsFixed(0)} =>
$crossAxisCount columns)',
              style: Theme.of(context).textTheme.titleMedium,
            ),
            const SizedBox(height: 12),
            GridView.builder(
              shrinkWrap: true,
              physics: const NeverScrollableScrollPhysics(),
              itemCount: 12,
              gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
                crossAxisCount: crossAxisCount,
                crossAxisSpacing: 12,
                mainAxisSpacing: 12,
                childAspectRatio: childAspectRatio,
              ),
              itemBuilder: (context, index) {
                return _ProductItemCard(
                  title: 'Product number $index with a quite long name',
                  price: '\$${(index + 1) * 10}',
                );
              },
            ),
          ],
        );
      },
    );
  }
}

class _ProductItemCard extends StatelessWidget {
  final String title;
  final String price;

  const _ProductItemCard({
    required this.title,
    required this.price,
  });

  @override
  Widget build(BuildContext context) {
    final double screenWidth = MediaQuery.of(context).size.width;

    // clamp trả về num -> ép sang double bằng toDouble()
    final double titleFontSize =
        (screenWidth * 0.03).clamp(12, 16).toDouble();
    final double priceFontSize =
        (screenWidth * 0.035).clamp(14, 18).toDouble();

    return Container(
      decoration: BoxDecoration(
        color: Colors.orange.withOpacity(0.1),
        borderRadius: BorderRadius.circular(12),
        border: Border.all(color: Colors.orange, width: 1),
```


PhucPT10@fpt.edu.vn

```
      ),
      padding: const EdgeInsets.all(8),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Tên sản phẩm dài → cần maxLines + ellipsis
          Text(
            title,
            style: TextStyle(
              fontSize: titleFontSize, // ✅ double
              fontWeight: FontWeight.w500,
            ),
            maxLines: 2,
            overflow: TextOverflow.ellipsis,
          ),
          const Spacer(),
          Text(
            price,
            style: TextStyle(
              fontSize: priceFontSize, // ✅ double
              fontWeight: FontWeight.bold,
              color: Colors.deepOrange,
            ),
          ),
        ],
      ),
    );
  }
}
```

7.3. Giải thích

`LayoutBuilder` **chỉ biết kích thước vùng nó chiếm**, không cần biết toàn màn hình.

Đây là khác biệt:

`MediaQuery`: Kích thước **toàn màn hình**.

`LayoutBuilder`: Kích thước **vùng widget** (cột bên trái, panel phải, v.v.)

`shrinkWrap: true` + `NeverScrollableScrollPhysics()` vì:

`GridView` nằm trong `SingleChildScrollView` của `DashboardScreen`.

Nếu cho `GridView` tự scroll nữa → conflict scroll, khó điều khiển.

8. BƯỚC 8 – KIỂM TRA TEXT OVERFLOW

Chúng ta đã có ví dụ trong `_ProductItemCard`:

Dart
```
Text(
  title,
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
);
```

Và trong `SummaryCard`:

Dart
```
Text(
  title,
  maxLines: 1,
```


PhucPT10@fpt.edu.vn

```
  overflow: TextOverflow.ellipsis,
);
```

8.1. Mục tiêu sư phạm

Sinh viên hiểu: **responsive không chỉ là số cột, width/height**.

Text cũng phải được:

Giới hạn số dòng (`maxLines`).

Định nghĩa cách xử lý tràn (`overflow: TextOverflow.ellipsis`).

Yêu cầu sinh viên tự thử:

Đổi title `SummaryCard` thành câu rất dài.

Xóa `maxLines` & `overflow` xem điều gì xảy ra (tràn, layout xấu).

Thêm lại để thấy khác biệt.

9. Kiểm tra trên nhiều màn hình – Checklist

9.1. Emulator đề xuất

Phone nhỏ: Pixel 3a / 4a (chiều ngang ~ 360–400)

Phone lớn: Pixel 6 Pro / 7 Pro

Tablet: bất kỳ (Android Tablet 10", 7", v.v.)

9.2. Checklist

Không có lỗi overflow (khung vàng + icon cảnh báo).

Summary cards:

Màn nhỏ: xếp dọc, full width.

Màn lớn: xếp ngang, 3 card chia đều.

Grid:

Màn nhỏ: 2 cột, chữ đọc được.

Màn vừa: 3 cột.

Màn to: 4 cột.

Text dài:

Không bị tràn khỏi card.

Có dấu `...` khi quá dài.

10. Tổng hợp mã nguồn hoàn chỉnh (tham khảo)

Đây là bản "hoàn chỉnh" tối thiểu cho dự án mẫu.

10.1. `lib/main.dart`

lib/main.dartv2

```dart
import 'package:flutter/material.dart';
import 'screens/dashboard_screen.dart';

void main() {
  runApp(const ResponsiveDashboardApp());
}

class ResponsiveDashboardApp extends StatelessWidget {
  const ResponsiveDashboardApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Responsive Dashboard',
```


PhucPT10@fpt.edu.vn

```
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: false,
      ),
      home: const DashboardScreen(),
    );
  }
}
```

10.2. lib/screens/dashboard_screen.dart

lib/screens/dashboard_screen.dartv4

```
import 'package:flutter/material.dart';
import '../widgets/summary_card.dart';
import '../widgets/responsive_grid.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final screenSize = MediaQuery.of(context).size;
    final screenWidth = screenSize.width;

    final horizontalPadding = screenWidth * 0.04;
    final headerFontSize = screenWidth * 0.06;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Responsive Dashboard'),
      ),
      body: SingleChildScrollView(
        child: Padding(
          padding: EdgeInsets.symmetric(horizontal: horizontalPadding),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: const [
              SizedBox(height: 16),

              _Header(),

              SizedBox(height: 16),

              _SummarySection(),

              SizedBox(height: 24),

              ResponsiveProductGrid(),
            ],
          ),
        ),
      ),
    );
  }
```


PhucPT10@fpt.edu.vn

```
}

class _Header extends StatelessWidget {
  const _Header();

  @override
  Widget build(BuildContext context) {
    final screenWidth = MediaQuery.of(context).size.width;
    final headerFontSize = (screenWidth * 0.06).clamp(20, 32);

    return Text(
      'Dashboard',
      style: TextStyle(
        fontSize: headerFontSize,
        fontWeight: FontWeight.bold,
      ),
    );
  }
}

class _SummarySection extends StatelessWidget {
  const _SummarySection();

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final isNarrow = constraints.maxWidth < 600;

        final cards = [
          const SummaryCard(
            title: 'Revenue',
            value: '\$12,300',
            color: Colors.blue,
          ),
          const SummaryCard(
            title: 'Orders',
            value: '320',
            color: Colors.green,
          ),
          const SummaryCard(
            title: 'Customers',
            value: '89',
            color: Colors.purple,
          ),
        ];

        if (isNarrow) {
          // Column cho màn nhỏ
          return Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              for (final card in cards)
```


PhucPT10@fpt.edu.vn

```
                Padding(
                  padding: const EdgeInsets.only(bottom: 12),
                  child: card,
                ),
            ],
          );
        } else {
          // Row + Expanded cho màn rộng
          return Row(
            children: [
              Expanded(child: cards[0]),
              const SizedBox(width: 12),
              Expanded(child: cards[1]),
              const SizedBox(width: 12),
              Expanded(child: cards[2]),
            ],
          );
        }
      },
    );
  }
}
```

10.3. `lib/widgets/summary_card.dart`

lib/widgets/summary_card.dartv2

```
import 'package:flutter/material.dart';

class SummaryCard extends StatelessWidget {
  final String title;
  final String value;
  final Color color;

  const SummaryCard({
    super.key,
    required this.title,
    required this.value,
    required this.color,
  });

  @override
  Widget build(BuildContext context) {
    // 1 Lấy chiều rộng màn hình (double)
    final double screenWidth = MediaQuery.of(context).size.width;

    // 2 Tính font size theo tỉ lệ và ép về double
    final double titleFontSize =
        (screenWidth * 0.035).clamp(12, 18).toDouble();
    final double valueFontSize =
        (screenWidth * 0.05).clamp(18, 28).toDouble();

    return Container(
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
```

```
        color: color.withOpacity(0.08),
        borderRadius: BorderRadius.circular(12),
        border: Border.all(color: color, width: 1),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Tiêu đề
          Text(
            title,
            style: TextStyle(
              fontSize: titleFontSize, // ✅ double
              fontWeight: FontWeight.w500,
              color: color,
            ),
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
          ),

          const SizedBox(height: 8),

          // Giá trị
          Text(
            value,
            style: TextStyle(
              fontSize: valueFontSize, // ✅ double
              fontWeight: FontWeight.bold,
              color: Colors.black87,
            ),
          ),
        ],
      ),
    );
  }
}
```

10.4. lib/widgets/responsive_grid.dart

lib/widgets/responsive_grid.dartv2

```
import 'package:flutter/material.dart';

class ResponsiveProductGrid extends StatelessWidget {
  const ResponsiveProductGrid({super.key});

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        // 1 Rõ kiểu double
        final double maxWidth = constraints.maxWidth;

        int crossAxisCount;
        if (maxWidth < 600) {
          crossAxisCount = 2;
```

```
        } else if (maxWidth < 900) {
          crossAxisCount = 3;
        } else {
          crossAxisCount = 4;
        }

        // 2 childAspectRatio là double hằng
        const double childAspectRatio = 1.1;

        return Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Products (width: ${maxWidth.toStringAsFixed(0)} →
$crossAxisCount columns)',
              style: Theme.of(context).textTheme.titleMedium,
            ),
            const SizedBox(height: 12),
            GridView.builder(
              shrinkWrap: true,
              physics: const NeverScrollableScrollPhysics(),
              itemCount: 12,
              gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
                crossAxisCount: crossAxisCount,
                crossAxisSpacing: 12,
                mainAxisSpacing: 12,
                childAspectRatio: childAspectRatio,
              ),
              itemBuilder: (context, index) {
                return _ProductItemCard(
                  title: 'Product number $index with a quite long name',
                  price: '\$${(index + 1) * 10}',
                );
              },
            ),
          ],
        );
      },
    );
  }
}

class _ProductItemCard extends StatelessWidget {
  final String title;
  final String price;

  const _ProductItemCard({
    required this.title,
    required this.price,
  });

  @override
  Widget build(BuildContext context) {
```


PhucPT10@fpt.edu.vn

```
    // 3 screenWidth là double
    final double screenWidth = MediaQuery.of(context).size.width;

    // 4 clamp trả về num → ép về double bằng toDouble()
    final double titleFontSize =
        (screenWidth * 0.03).clamp(12, 16).toDouble();
    final double priceFontSize =
        (screenWidth * 0.035).clamp(14, 18).toDouble();

    return Container(
      decoration: BoxDecoration(
        color: Colors.orange.withOpacity(0.08),
        borderRadius: BorderRadius.circular(12),
        border: Border.all(color: Colors.orange, width: 1),
      ),
      padding: const EdgeInsets.all(8),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Tên sản phẩm dài → cần maxLines + ellipsis
          Text(
            title,
            style: TextStyle(
              fontSize: titleFontSize, // ✅ double
              fontWeight: FontWeight.w500,
            ),
            maxLines: 2,
            overflow: TextOverflow.ellipsis,
          ),
          const Spacer(),
          Text(
            price,
            style: TextStyle(
              fontSize: priceFontSize, // ✅ double
              fontWeight: FontWeight.bold,
              color: Colors.deepOrange,
            ),
          ),
        ],
      ),
    );
  }
}
```

11. Gợi ý bài tập mở rộng

Sau khi hoàn thành:

Thêm breakpoint tablet lớn

Ví dụ: > 1200px → chia layout thành 2 cột: bên trái summary, bên phải grid.

Xử lý khi xoay ngang (landscape)

Dùng `MediaQuery.of(context).orientation` để thay đổi layout.

Chọn một màn hình khác trong app khác (ví dụ: màn hình profile / settings)

Yêu cầu sinh viên refactor theo đúng tư duy:

PhucPT10@fpt.edu.vn

Tách widget,
Dùng LayoutBuilder cho layout chính,
Dùng MediaQuery cho spacing/font.

PhucPT10@fpt.edu.vn

🧠 NHÓM 1. TƯ DUY & KHÁI NIỆM NỀN TẢNG

Thuật ngữ	Giải thích dễ hiểu	Ý nghĩa trong Flutter thực tế
Responsive UI	UI tự điều chỉnh bố cục theo kích thước & môi trường thiết bị	1 UI chạy ổn trên nhiều màn hình (phone, tablet, resize)
Adaptive UI	UI thay đổi **cấu trúc** theo nền tảng (mobile / tablet / desktop)	Dùng layout khác nhau cho thiết bị khác nhau
UI cố định (Fixed UI)	UI dùng size cứng (width/height cụ thể)	Nguyên nhân chính gây vỡ layout
UI linh hoạt (Flexible UI)	UI dựa trên không gian được cấp	Mục tiêu cần đạt khi làm Flutter thật
Pixel cứng (Hard-coded pixel)	Gán số cụ thể: 200, 300, 16px	Anti-pattern trong Flutter
Breakpoint	Mốc kích thước để đổi layout	Phone / Tablet / Desktop
Multi-device UI	UI chạy trên nhiều thiết bị	Thực tế khi release app

🎓 **Sinh viên hay hiểu sai**:
"Responsive = MediaQuery" ❌
→ Thực chất responsive là **tư duy layout theo constraint**

📐 NHÓM 2. FLUTTER LAYOUT ENGINE (CỐT LÕI)

Thuật ngữ	Giải thích	Ý nghĩa thực tế
Constraint	Giới hạn không gian cha cấp cho con	Flutter layout không tự do
Layout constraint system	Cơ chế layout của Flutter	Hiểu được thì debug UI rất dễ
Parent widget	Widget bao ngoài	Quyết định con được to cỡ nào
Child widget	Widget bên trong	Phải tuân theo constraint
Unbounded constraint	Không có giới hạn size	Hay gây lỗi với ListView
Bounded constraint	Có giới hạn rõ ràng	An toàn cho layout
Size negotiation	Quá trình widget chọn kích thước	Không phải pixel tuyệt đối
Positioning	Cha đặt con ở đâu	Alignment, padding

📌 **Câu thần chú Flutter**:
Parent → constraints → Child → size → Parent → position

📱 NHÓM 3. MEDIAQUERY & KÍCH THƯỚC MÀN HÌNH

Thuật ngữ	Giải thích	Dùng trong app thật
MediaQuery	Lấy thông tin môi trường thiết bị	Screen size, orientation
MediaQuery.of(context)	Cách truy cập MediaQuery	Thường dùng ở screen level
Screen size	Kích thước toàn màn hình	Không phải widget size
Orientation	Dọc / Ngang	Phone xoay ngang
Device padding	Notch, status bar	Tránh UI bị che
viewInsets	Bàn phím che UI	Form bị đẩy lên
SafeArea	Widget tránh vùng nguy hiểm	App production bắt buộc
Global size	Size toàn app	Dễ bị lạm dụng

⚠️ **Cảnh báo**:
MediaQuery **không biết widget con đang rộng bao nhiêu**

PhucPT10@fpt.edu.vn

📦 NHÓM 4. LAYOUTBUILDER & CONSTRAINT-BASED DESIGN

Thuật ngữ	Giải thích	Giá trị thực tế
LayoutBuilder	Widget cho biết constraint hiện tại	Responsive đúng nghĩa
BoxConstraints	Thông tin giới hạn kích thước	maxWidth / maxHeight
constraints.maxWidth	Độ rộng tối đa widget được phép	Quyết định layout bên trong
Local constraint	Constraint của widget hiện tại	Chuẩn cho component
Reusable component	Widget tái sử dụng	Cần LayoutBuilder
Contextual layout	Layout phụ thuộc vị trí	Không phụ thuộc screen

🎓 **Sinh viên thường sai**:
Dùng MediaQuery trong widget con ❌
→ Đúng ra nên dùng **LayoutBuilder**

🧩 NHÓM 5. WIDGET RESPONSIVE CƠ BẢN

Thuật ngữ	Giải thích	Dùng khi nào
Row	Layout ngang	Toolbar, action
Column	Layout dọc	Form, list
Expanded	Chiếm hết không gian còn lại	Chia layout
Flexible	Chiếm không gian linh hoạt	Tránh overflow
Spacer	Khoảng trống linh hoạt	Thay vì SizedBox
IntrinsicHeight/Width	Tính size dựa nội dung	Dùng rất hạn chế
Wrap	Tự xuống dòng	Button, tag
AspectRatio	Giữ tỉ lệ	Card, image

📌 **Lưu ý**:
Expanded ≠ Responsive
Expanded chỉ là **công cụ chia không gian**

🗔 NHÓM 6. GRIDVIEW & LIST RESPONSIVE

Thuật ngữ	Giải thích	Thực tế
GridView	Layout dạng lưới	Product, gallery
SliverGridDelegate	Quy tắc chia grid	Quyết định cột
crossAxisCount	Số cột	Dễ gây cứng
maxCrossAxisExtent	Chiều rộng tối đa item	Responsive tốt hơn
ListView	Danh sách 1 chiều	Feed, chat
Scrollable overflow	Tràn khi scroll	Lỗi phổ biến

abc NHÓM 7. TEXT, SPACING & OVERFLOW

Thuật ngữ	Giải thích	Ý nghĩa
Text overflow	Text tràn layout	UI vỡ không báo lỗi
TextOverflow.ellipsis	Dấu . . .	Bắt buộc cho list
Soft wrap	Text tự xuống dòng	Responsive text
Line limit (maxLines)	Giới hạn dòng	Tránh vỡ UI
Responsive typography	Font linh hoạt	Không scale pixel
Spacing	Khoảng cách	Dùng constraint thay vì số

🔧 NHÓM 8. RESPONSIVE REFACTOR & DEBUG

Thuật ngữ	Giải thích	Khi đi làm
Refactor UI	Cải tổ layout	Kỹ năng bắt buộc
Layout smell	Dấu hiệu UI xấu	Code khó mở rộng
Overflow warning	Cảnh báo UI tràn	Debug layout

PhucPT10@fpt.edu.vn

Visual debug	Bật debug paint	Tìm lỗi nhanh
Hot reload illusion	UI đúng lúc dev	Sai trên máy khác
Technical debt UI	Nợ kỹ thuật giao diện	Rất khó sửa về sau

3 THUẬT NGỮ SINH VIÊN PHẢI NHỚ NHẤT

Constraint – hiểu cái này là hiểu Flutter layout
Local size ≠ Screen size
Responsive là tư duy, không phải widget

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

🔍 PROMPT 1

Phân tích sự khác biệt giữa responsive theo Screen size và responsive theo Constraint trong Flutter.

👉 Gợi ý nghiên cứu:
MediaQuery vs LayoutBuilder
Trường hợp nào hai cách cho kết quả khác nhau?
Ảnh hưởng tới khả năng tái sử dụng widget
Đánh giá mức độ phù hợp trong dự án lớn

🎯 Mục tiêu:
→ Hiểu vì sao **LayoutBuilder là nền tảng**, MediaQuery chỉ là công cụ hỗ trợ

🔍 PROMPT 2

Đánh giá hậu quả dài hạn của việc hard-code kích thước UI trong Flutter project quy mô vừa.

👉 Phân tích:
UI vỡ ở giai đoạn nào của dự án?
Refactor tốn bao nhiêu chi phí?
Những đoạn code nào "đáng sợ" nhất khi mở rộng thiết bị?

🎯 Mục tiêu:
→ Nhận diện **nợ kỹ thuật UI (UI technical debt)**

🔍 PROMPT 3

So sánh hiệu quả của các chiến lược responsive layout: Expanded/Flexible vs GridView vs Wrap.

👉 Yêu cầu:
Mỗi chiến lược phù hợp với loại màn hình nào?
Khi nào Expanded gây overflow?
Khi nào GridView làm UI khó maintain?

🎯 Mục tiêu:
→ Không dùng widget theo cảm tính

🔍 PROMPT 4

Phân tích vai trò của LayoutBuilder trong việc xây dựng reusable component trong Flutter.

👉 Gợi ý:
Vì sao widget dùng MediaQuery khó tái sử dụng?
LayoutBuilder giúp component độc lập thế nào?
Ví dụ: Card, Item, Tile

🎯 Mục tiêu:
→ Tiếp cận tư duy **component-driven design**

🔍 PROMPT 5

Đánh giá tác động của responsive UI đến khả năng maintain và scale Flutter app trong 2–3 năm.

👉 Phân tích:
Thêm màn hình mới
Thêm thiết bị mới
Thay đổi UX theo yêu cầu khách hàng

🎯 Mục tiêu:
→ Nhìn responsive UI như **chiến lược kiến trúc**, không chỉ là UI

🔍 PROMPT 6

Phân tích các lỗi responsive UI "không báo lỗi" trong Flutter và vì sao chúng khó debug.

PhucPT10@fpt.edu.vn

👉 Gợi ý:
Text overflow
Unbounded height
Nested scroll
Layout vỡ khi xoay màn hình
🎯 Mục tiêu:
→ Nâng cao kỹ năng debug layout thực tế

🔍 PROMPT 7
So sánh tư duy responsive UI trong Flutter với responsive trong HTML/CSS (Flexbox, Grid).
👉 Phân tích:
Điểm giống về layout flow
Khác biệt về constraint & measurement
Vì sao dev web dễ sai khi chuyển sang Flutter?
🎯 Mục tiêu:
→ Tránh "áp tư duy web sai chỗ"

🔍 PROMPT 8
Thiết kế hệ thống breakpoint đơn giản cho Flutter app và đánh giá ưu/nhược điểm.
👉 Yêu cầu:
Bao nhiêu breakpoint là hợp lý?
Breakpoint theo screen hay theo constraint?
Tách widget theo breakpoint có gây phức tạp không?
🎯 Mục tiêu:
→ Biết **thiết kế breakpoint**, không copy máy móc

🔍 PROMPT 9
Phân tích quy trình refactor một màn hình Flutter từ fixed UI sang responsive UI.
👉 Trình bày:
Nhận diện layout cứng
Phân tích constraint
Quyết định dùng MediaQuery hay LayoutBuilder
Đánh giá kết quả sau refactor
🎯 Mục tiêu:
→ Thành thạo kỹ năng refactor – kỹ năng sống còn khi đi làm

🔍 PROMPT 10
Đánh giá khi nào responsive UI là chưa đủ và cần chuyển sang adaptive UI trong Flutter.
👉 Gợi ý:
Tablet layout khác mobile
Desktop resize liên tục
UX khác biệt theo thiết bị
🎯 Mục tiêu:
→ Chuẩn bị tư duy cho **slot tiếp theo: Adaptive UI & Advanced Layout**

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

PROMPT 1

Thiết kế hệ thống responsive layout cho một Flutter app có hơn 30 màn hình.

👉 Sinh viên cần phân tích:
Có nên áp dụng responsive ngay từ đầu hay refactor dần?
Nên đặt breakpoint ở đâu trong Widget Tree?
Cách tổ chức folder để responsive không bị rối?

🎯 Góc nhìn chuyên nghiệp:
→ Tư duy kiến trúc UI, không phải từng màn hình đơn lẻ

PROMPT 2

Đánh giá tác động của responsive UI tới khả năng mở rộng sản phẩm trong 1–2 năm.

👉 Phân tích:
Khi thêm tablet / màn hình lớn
Khi thay đổi thiết kế UI
Khi đội UX thay đổi flow

🎯 Mục tiêu:
→ Hiểu responsive UI là **đầu tư dài hạn**

PROMPT 3

Xử lý trường hợp UI ổn trên emulator nhưng vỡ trên máy khách hàng thật.

👉 Sinh viên cần tìm hiểu:
Các yếu tố gây sai khác (density, safe area, font scale)
Cách test thực tế
Chiến lược phòng tránh

🎯 Thực tế đi làm:
→ Bug UI thường xuất hiện ở **máy của khách**

PROMPT 4

Khi nào MediaQuery trở thành "mùi code xấu" (code smell) trong dự án Flutter?

👉 Phân tích:
Dùng MediaQuery quá nhiều gây vấn đề gì?
Dấu hiệu cần refactor sang LayoutBuilder
Ảnh hưởng tới test & reuse widget

🎯 Tư duy senior:
→ Biết khi nào **không nên dùng**

PROMPT 5

Thiết kế reusable UI component hoạt động tốt trên nhiều kích thước màn hình.

👉 Ví dụ:
Card
Product item
List tile

👉 Câu hỏi:
Nên dùng constraint hay screen size?
Component có nên tự responsive không?

🎯 Chuẩn công ty:
→ Component phải "sống độc lập"

PhucPT10@fpt.edu.vn

📦 PROMPT 6

Phân tích rủi ro khi refactor responsive UI quá muộn trong dự án Flutter.

👉 Nghiên cứu:
Chi phí sửa code
Rủi ro bug
Ảnh hưởng tới tiến độ release

🎯 Bài học xương máu:
→ Responsive làm muộn = trả giá đắt

📊 PROMPT 7

Thiết kế GridView hiển thị đúng trên mobile nhỏ, mobile lớn và tablet.

👉 Sinh viên cần:
Phân tích số cột theo constraint
So sánh GridView.count vs GridView.builder
Xử lý spacing & aspect ratio

🎯 Thực tế:
→ Grid là nơi UI vỡ nhiều nhất

🧪 PROMPT 8

Xây dựng chiến lược test responsive UI cho Flutter app.

👉 Gợi ý:
Test thủ công vs test tự động
Golden test có giúp được gì?
Test theo breakpoint hay thiết bị?

🎯 Góc nhìn chuyên nghiệp:
→ Responsive không test = may rủi

🧠 PROMPT 9

Phân tích sự khác nhau giữa responsive UI cho MVP và cho sản phẩm thương mại.

👉 So sánh:
Mức độ đầu tư
Độ phức tạp
Chi phí refactor

🎯 Tư duy sản phẩm:
→ Không phải app nào cũng cần full responsive ngay

📦 PROMPT 10

Thiết kế layout có thể resize liên tục (window resize) trong Flutter.

👉 Áp dụng cho:
Desktop
Web
Tablet xoay ngang

🎯 Chuẩn nâng cao:
→ Responsive theo constraint động

🧠 PROMPT 11

Đánh giá việc tách widget theo breakpoint: lợi ích và rủi ro.

👉 Phân tích:
Code dễ đọc hơn hay phức tạp hơn?
Ảnh hưởng tới performance?
Khi nào nên / không nên làm?

PhucPT10@fpt.edu.vn

🎯 Góc nhìn lead:
→ Đánh đổi giữa clarity & complexity

📦 PROMPT 12
Xử lý Text, font scale và overflow trong UI đa thiết bị.

👉 Nghiên cứu:
TextScaleFactor
Overflow ellipsis
Auto-wrap

🎯 Thực tế:
→ Text là thứ vỡ nhiều nhất nhưng hay bị bỏ qua

🧠 PROMPT 13
Phân tích vai trò của Responsive UI trong UX tổng thể của Flutter app.

👉 Sinh viên cần đánh giá:
Dễ dùng hơn ở đâu?
Khó dùng hơn ở đâu?
Responsive sai ảnh hưởng UX thế nào?

🎯 Tư duy sản phẩm:
→ UI đẹp nhưng dùng khó = thất bại

📦 PROMPT 14
So sánh chiến lược responsive UI trong Flutter mobile-only và Flutter đa nền tảng.

👉 Phân tích:
Mobile-only cần gì?
Khi lên web/desktop cần thay đổi gì?
Chiến lược chuyển đổi

🎯 Chuẩn bị tương lai:
→ Flutter không chỉ cho mobile

🧠 PROMPT 15
Đóng vai Tech Lead: Review responsive UI của một Flutter app và đề xuất cải tiến.

👉 Yêu cầu:
Nhận diện lỗi tư duy
Đánh giá rủi ro
Đề xuất refactor

🎯 Mục tiêu cao nhất:
→ Tập suy nghĩ như **người chịu trách nhiệm kỹ thuật**

PhucPT10@fpt.edu.vn